

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 19 July 2017 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

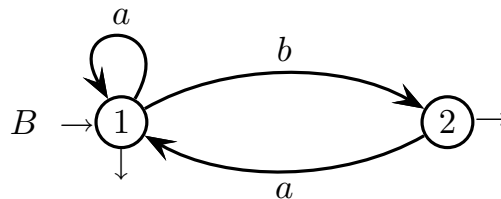
1 Regular Expressions and Finite Automata 20%

1. Consider the two-letter alphabet $\Sigma = \{ a, b \}$ and answer the following questions:

- (a) By using the Berry-Sethi method, design a deterministic finite-state automaton A that accepts the language generated by the regular expression R_A below:

$$R_A = (a b)^* (b a a)^*$$

- (b) Check if the automaton A designed at point (a) is minimal, and minimize it if necessary.
- (c) Consider the finite-state automaton B shown below, with initial state 1, and two final states 1 and 2:



By using the *BMC* method (node elimination), write a regular expression R_B equivalent to the above automaton B .

- (d) From the automaton B of point (c) write, in a systematic way, an equivalent right-linear grammar and a system of linear language equations in the unknowns L_1 and L_2 , which respectively represent the languages associated to the states 1 and 2 of B .

Then solve the equation system and write a regular expression R_1 for L_1 (make sure that expression R_1 is equivalent to the expression R_B of point (c)).

- (e) (optional) By using a systematic method, design a deterministic finite-state automaton that accepts the intersection language $L(R_A) \cap L(B)$.

2 Free Grammars and Pushdown Automata 20%

1. Consider the following two context-free languages L_1 and L_2 , over the two-letter alphabet $\Sigma = \{ a, b \}$:

$$L_1 = \{ a^{h_1} b^{k_1} \mid k_1 \geq h_1 \geq 0 \}$$

$$L_2 = \{ b^{k_2} a^{h_2} \mid k_2 > h_2 \geq 1 \}$$

Answer the following questions:

- (a) Write two *BNF* grammars G_1 and G_2 (no matter if they are ambiguous) that generate languages L_1 and L_2 , respectively, and test such grammars by drawing the syntax trees of the valid strings $abb \in L_1$ and $bba \in L_2$.
 - (b) Consider the concatenation language $L_3 = L_1 \cdot L_2$, write a description of L_3 as a set of strings (in the style of the above definitions of L_1 and L_2), and write a *BNF* grammar G_3 that generates language L_3 , by using as components the grammars G_1 and G_2 found at point (a).
 - (c) (optional) Say if the grammar G_3 found at point (b) is ambiguous or not. If it is ambiguous, say if the language L_3 is inherently ambiguous or not, and justify your answer.
-

2. Consider a fragment of a programming language that allows a user to define arithmetic expressions by using the well known “let *var* in *expr*” construct, where the value of a nested expression *expr* is determined by that of the variables defined in the *var* clause of the enclosing expressions. A language phrase consists of a single expression introduced by the keyword let. Below are the phrase composition rules:

- The *var* clauses contain one or more variable definitions, separated by a comma “,”; each definition uses the operator “=” with a variable name in the left part and an arithmetic expression in the right part.
- Arithmetic expressions can contain arithmetic subexpressions of the type *if-then-else-endif*, the condition of which is composed of a relational operator (namely, “==” for equality, “!=” for inequality, and “>” and “≥” with the usual meaning) applied to two arithmetic expressions.
- Arithmetic expressions can contain the usual operators of addition “+”, subtraction “−” (possibly unary), multiplication “*” and division “/”. The operator precedences are as usual, and the *if-then-else-endif* subexpressions have the highest precedence. Notice that parenthesized subexpressions are allowed.
- The atomic subexpressions are identifiers and numbers, represented by the terminals *id* and *num*, respectively.

Further detailed information about the lexical and syntactic structure of the programming language can be inferred from the example below:

```
let (x = 3, y = 7) in

  y - let (z = 9 - y) in

    x + z * if (z * 2 == 17)

      then 4 * (y + x)

    else

      let (alpha = -z / 2) in

        alpha - y / x

      endlet

    endif

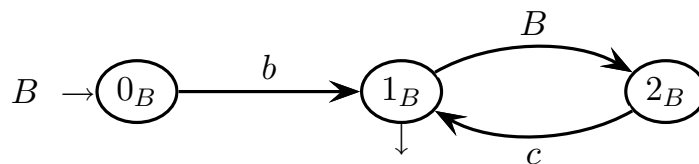
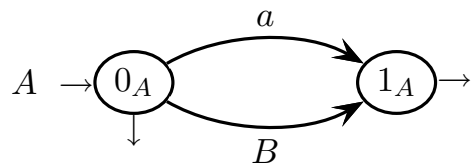
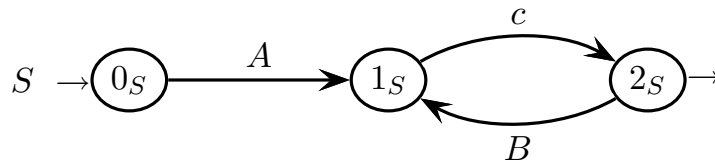
  endlet

endlet
```

Write a grammar, *EBNF* and unambiguous, that models the sketched programming language fragment.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the three-letter terminal alphabet $\Sigma = \{ a, b, c \}$ and the three-letter nonterminal alphabet $V = \{ S, A, B \}$ (axiom S):

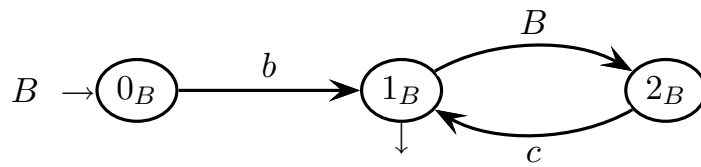
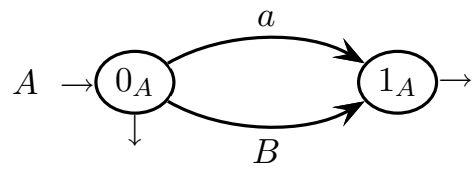
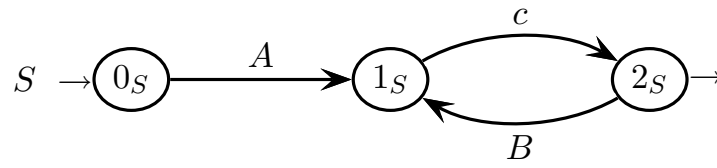


Answer the following questions:

- (a) Draw the complete pilot of grammar G , say if grammar G is of type $ELR(1)$, and shortly justify your answer (highlight all the conflicts, if any).
- (b) Write the necessary guide sets on all the arcs of the machine net (call arcs and exit arrows), say if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer (highlight all the conflicts, if any).
- (c) Independently of whether grammar G is of type $ELL(1)$, it has at least one machine without conflicting guide sets. Write the recursive descent syntactic procedure of such a machine (if there are two or more, choose one at will).
- (d) (optional) Examine the language $L(G)$ and determine if it can be parsed with a recognizer type simpler than a pushdown automaton (of whatever kind).

please here draw the complete pilot of grammar G

please here draw the call arcs and write all the guide sets



4 Language Translation and Semantic Analysis 20%

1. The *BNF* source grammar G_S below (axiom S), over the two-letter terminal alphabet $\{ a, b \}$:

$$G_S \begin{cases} S \rightarrow a a b \\ S \rightarrow a a a S b b \end{cases}$$

generates the source language L_S below:

$$L_S = \{ a^i b^j \mid 2i = 3j + 1 \}$$

The language L_S includes, among others, the following three strings:

$$a^2 b \quad a^5 b^3 \quad a^8 b^5$$

Please answer the following questions:

- (a) Write a target (or destination) grammar G_T associated to the source grammar G_S , without changing G_S , that generates the following target language L_T :

$$L_T = \{ a^i c b^j \mid 3i = 2j + 2 \}$$

according to the four translation examples below:

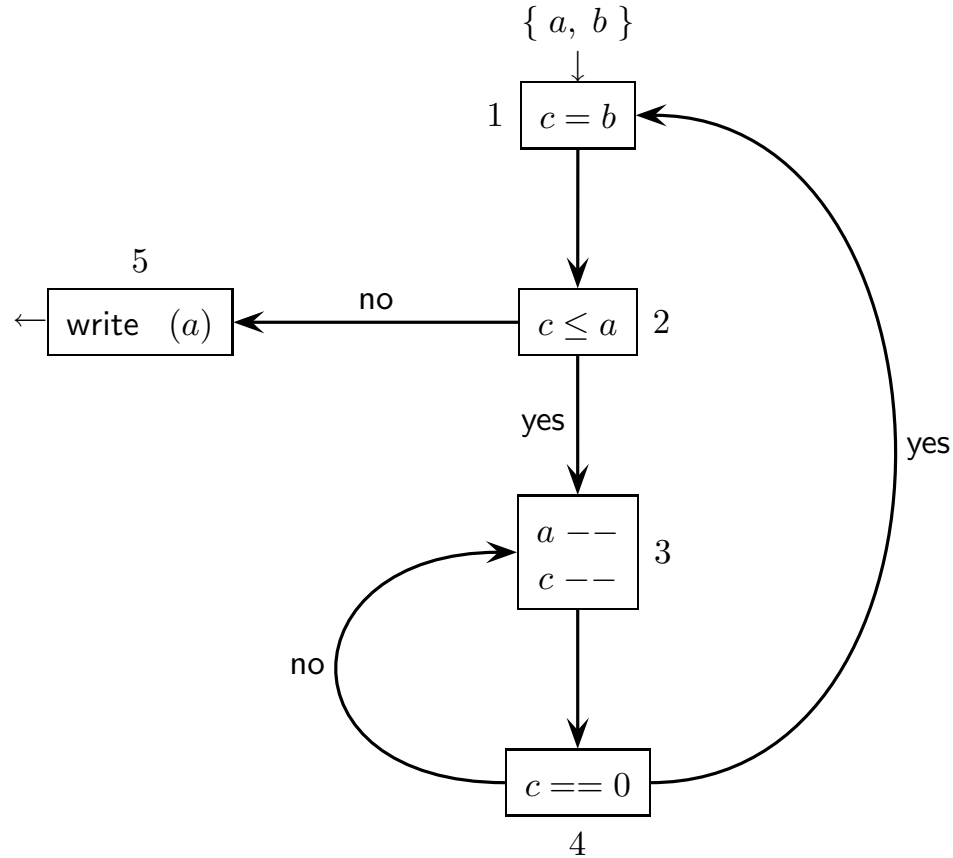
$$a^2 b \mapsto a^2 c b^2 \quad a^5 b^3 \mapsto a^4 c b^5 \quad a^8 b^5 \mapsto a^6 c b^8 \quad a^{11} b^7 \mapsto a^8 c b^{11}$$

- (b) Write the syntax trees of the source and target strings for the two translation examples below:

$$a^2 b \mapsto a^2 c b^2 \quad a^5 b^3 \mapsto a^4 c b^5$$

- (c) (optional) Can the translation defined by the scheme above be computed deterministically? Provide an adequate justification to your answer.

2. Consider the following Control Flow Graph (CFG) of a program, with some nodes that contain multiple assignments (input node 1 and output node 5):



This program has two input parameters a and b , and one variable c . All of them are integers, with $a \geq 0$ and $b \geq 1$.

Answer the following questions:

- Find a regular expression R , over the node name alphabet, that describes the execution traces of the program, by ignoring any semantic restriction.
- Write the system of flow equations for the *reaching definitions* of the CFG, and iteratively solve the equation system (use the tables and the figure prepared on the next pages).
- (optional) Determine the final value of a written by the program as a function of the initial values of a and b . Based on this function, refine the regular expression R found at point (a) into a language expression, over the node name alphabet, that exactly defines the program execution traces. Is the language of the execution traces still regular? Adequately justify your answer.

table of definitions and suppressions at the nodes
for the **REACHING DEFINITIONS**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>suppressed</i>
1		1	
2		2	
3		3	
4		4	
5		5	

system of data-flow equations for the **REACHING DEFINITIONS**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		

iterative solution table of the system of data-flow equations (**REACHING DEF.S**)
(the number of columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														

write here the **REACHING DEFINITIONS** computed

